

LTIM_AIServicesB2_MS2Set1_Q2

Test Summary

No. of Sections: 1
No. of Questions: 1
Total Duration: 900 min

Section 1 - COD

Section Summary

- No. of Questions: 1
- Duration: 900 min

Additional Instructions:

None

Q1. MACHINE CONDITION FAILURE RISK BANDING

PROBLEM STATEMENT

A precision manufacturing plant schedules preventive maintenance around production commitments. A machine that fails during a live order can scrap work in progress, stop an entire line and force an emergency maintenance call, so the planning desk would rather service a doubtful machine before the next production cycle than discover the fault during a shift.

Every machine is inspected before the maintenance roster is frozen and given a **failure risk band**. The judgement rests on several readings at once - how long the machine has run, how long since it was serviced, what its vibration and bearing temperature are showing, and how heavily it is worked each day. An assessment stage is required to learn the bands from machines already inspected, to confirm that the model chosen for the job is the better of the two considered, and to band the machines still waiting for the next roster.

OBJECTIVE

Build a **six-stage pipeline** that prepares the condition-monitoring exports for modelling, fits **two classifiers** and stores each of them, ranks the two against machines held back from fitting, measures the chosen classifier on those held-back machines, and applies the stored classifier to a fresh set of machines awaiting assessment.

FILE STRUCTURE

```
Project
|-- data
|   |-- machine_condition_train.csv
|   |-- machine_condition_test.csv
|   |-- machine_condition_predict.csv
|-- processed_data
|-- artifacts
|-- output
|-- main.py
`-- requirements.txt
```

PURPOSE OF FILES

Path	Purpose
data/machine_condition_train.csv	Banded assessments used to fit the classifiers.
data/machine_condition_test.csv	Banded assessments held back from fitting, used to rank and measure.
data/machine_condition_predict.csv	Machines awaiting a band.
processed_data	Destination for the model-ready tables.
artifacts	Destination for the two fitted classifiers.
output	Destination for the issued bands.
main.py	Module to be implemented.
requirements.txt	Package versions required by the pipeline.

DATASET DESCRIPTION

Training Dataset: machine_condition_train.csv

Column	Role	Description
machine_id	Record	Reference for the machine, for example MC-TR-0001286.
inspection_date	Record	Date the condition inspection was carried out.
plant_location	Record	Plant where the machine is installed.
production_line	Record	Production line the machine serves.
shift_engineer	Record	Engineer responsible for the shift.
maintenance_order_ref	Record	Maintenance work-order reference.
operating_hours_thousands	Feature	Lifetime running hours in thousands, from 0.3 to 18.0.
days_since_service	Feature	Days since the last planned service, from 5 to 720.
vibration_mm_s	Feature	Measured vibration velocity in mm/s, from 0.5 to 14.0. A rising figure can indicate wear.
bearing_temperature_celsius	Feature	Bearing temperature in Celsius, from 38 to 112.
average_daily_runtime_hours	Feature	Typical running time per day, from 4 to 23 hours.
machine_type	Feature	Machine family: cnc_lathe, milling_center or hydraulic_press.
load_profile	Feature	Typical duty: light, normal, heavy or cyclic.
lubrication_system	Feature	Lubrication arrangement: manual or automatic.
failure_risk_band	Target	Inspection band: VERY_LOW, LOW, MODERATE, HIGH or CRITICAL.

SAMPLE RECORDS

Column	Record 1	Record 2	Record 3
machine_id	MC-TR-0000001	MC-TR-0000002	MC-TR-0000003
inspection_date	2026-04-10 00:00:00	2026-07-20 00:00:00	2026-02-01 00:00:00
plant_location	Pune	Hyderabad	Hyderabad
production_line	Line III	Line II	Line III
shift_engineer	A-Khan	A-Khan	P-Rao
maintenance_order_ref	MO-866975	MO-132164	MO-938881
operating_hours_thousands	11.64	14.34	1.12
days_since_service	261	635	371
vibration_mm_s	10.22	(blank)	5.33
bearing_temperature_celsius	106.0	56.2	111.2
average_daily_runtime_hours	16.9	9.4	15.6
machine_type	cnc_lathe	hydraulic_press	MILLING_CENTER
load_profile	light	light	heavy
lubrication_system	manual	automatic	manual
failure_risk_band	CRITICAL	moderate	HIGH

machine_condition_test.csv holds rows of exactly this shape. machine_condition_predict.csv holds rows of the same shape with the final line, **failure_risk_band**, absent.

TASK

Implement the following functions and the main block described below inside main.py.

FUNCTION SIGNATURE

```
clean_data(input_path, output_path)
```

PARAMETERS

- **input_path** - string locating a condition-monitoring export inside data.
- **output_path** - string locating the model-ready table to be written in processed_data.

REQUIREMENTS

- Drop the six record columns. Keep the eight feature columns and failure_risk_band where present.
- Remove repeated records, keeping the first occurrence. This must happen before any figure is measured from a column.
- Fill blanks in vibration_mm_s, bearing_temperature_celsius and average_daily_runtime_hours with the median of their own column, not the mean.
- Normalise case and stray spaces in the text columns before encoding.
- Encode machine_type as cnc_lathe 0, milling_center 1, hydraulic_press 2.
- Encode load_profile as light 0, normal 1, heavy 2, cyclic 3.
- Encode lubrication_system as manual 0, automatic 1.
- Encode failure_risk_band, where present, as VERY_LOW 0, LOW 1, MODERATE 2, HIGH 3, CRITICAL 4.
- Leave no text column: every column that survives must come out numeric. Do not create failure_risk_band on an export that lacks it.
- Write to output_path with no index column.

RETURNS

pandas.DataFrame - the cleaned table.

FUNCTION SIGNATURE

```
train_logistic_model(processed_path, model_path)
```

REQUIREMENTS

- Fit a LogisticRegression with max_iter=1000, on every row of the file named by processed_path.
- Separate failure_risk_band from the columns fitted on.
- The columns span thousands of hours, days, mm/s, Celsius and hours a day. Every column must reach the classifier standardised to mean 0 and unit variance.
- The standardisation must be learned from the training machines only and stored together with the classifier, so the same shift and spread reapply later.
- Persist the stored object to model_path so it reloads without refitting.

RETURNS

A fitted scikit-learn estimator - one that bands cleaned assessments directly, carrying whatever preparation it needs inside itself.

FUNCTION SIGNATURE

```
train_svc_model(processed_path, model_path)
```

REQUIREMENTS

- Fit an SVC with kernel="rbf", C=1.0 and random_state=42, on every row of the file named by processed_path.
- Separate failure_risk_band from the columns fitted on.
- An SVC judges machines by the distance between them, so every column must reach it standardised to mean 0 and unit variance.
- The standardisation must be learned from the training machines only and stored together with the classifier.
- Persist the stored object to model_path so it reloads without refitting.

FUNCTION SIGNATURE

`compare_models(logistic_model_path, svc_model_path, features_test, target_test)`

REQUIREMENTS

- Load both classifiers from the paths given. Do not refit either.
- Score each on the machines passed in, not on any file read from disk.
- Report accuracy and F1 score for each. Across five bands of unequal size the F1 score must be averaged across the bands without weighting by their size.
- Return a table with columns `model_name`, `accuracy`, `f1_score` - one row per classifier, ordered by accuracy, strongest first.
- Name the models `LogisticRegression` and `SVC`.
- Nothing is written to disk by this stage.

RETURNS

`pandas.DataFrame` - the comparison table.

FUNCTION SIGNATURE

`evaluate_model(model_path, features_test, target_test)`

REQUIREMENTS

- Load the classifier from `model_path`. Do not refit it.
- Score it against the machines passed in.
- Report the accuracy and the F1 score, averaging the F1 score across the bands without weighting.

RETURNS

tuple of two float values, in the order listed above.

FUNCTION SIGNATURE

`predict_new_data(model_path, input_path, processed_path, output_path)`

REQUIREMENTS

- Load the classifier from `model_path`. Do not refit it.
- Prepare the machines at `input_path` with the same stage used for the training assessments, writing that prepared form to `processed_path`.
- The `machine_id` is removed during preparation, so read it from the unassessed export and keep it paired with the band issued for that same machine.
- Every machine receives exactly one band, recorded as the band name, not the code number.
- Write `machine_id` and `predicted_failure_risk_band` only to `output_path`, with no index column.

RETURNS

`pandas.DataFrame` - the banded machines.

MAIN BLOCK

- Runs the six stages in sequence and is the only place that prints.
- Prepares the training export and the held-back export, then separates the held-back machines into their feature columns and recorded bands.
- Fits and stores both classifiers, then ranks them.
- Reads the model named in the first row of the comparison table and passes that classifier's stored path into the measurement stage and the banding stage.
- Reports the shape of the cleaned training table, the shape of the cleaned held-back table, confirmation that both classifiers were stored, the comparison table, the accuracy and F1 score, and the shape of the banded machines.

GENERATED FILES

Path	Contents
processed_data/machine_condition_train_cleaned.csv	Model-ready training assessments with their bands.
processed_data/machine_condition_test_cleaned.csv	Model-ready held-back assessments with their bands.
processed_data/machine_condition_predict_cleaned.csv	Model-ready unassessed machines.
artifacts/logistic_regression_model.pkl	Fitted linear-boundary model.
artifacts/svc_model.pkl	Fitted support vector classifier.
output/machine_failure_risk_predictions.csv	Band issued for each unassessed machine.

EXPECTED OUTPUT

The exact scores are produced by the supplied data and may vary only slightly between compatible library versions. The required row counts and output shapes are:

```
Cleaned training assessments: (27000, 9)
Cleaned held-back assessments: (7715, 9)
Models trained and saved.
Model comparison:
      model_name  accuracy  f1_score
      SVC          ...         ...
  LogisticRegression  ...         ...
Accuracy: .... | Macro F1: ....
Predictions written: (800, 2)
```

The row counts fall below the counts in the exports because repeated records are removed.

IMPLEMENTATION STEPS

STEP 1 - Move into the project directory.
`cd Project`

STEP 2 - Install the pinned packages.
`pip install -r requirements.txt`

STEP 3 - Run the pipeline.
`python3 main.py`

STEP 4 - Run the testcases.
`python3 -m pytest ../pytest/tests.py -v`

Answer Key & Solution

Section 1 - COD

Q1 Solution cannot be displayed